POLYTECH
NICE SOPHIA

Equipe O

Conception logicielle

SI4 2025/2026

Rendu D1

UNIVERSITÉ
CÔTE D'AZUR

BOULET Olivier - PO

KARBAL Adam - OPS

MADANDAR ARANI Ali – PO & QA

MENOU Clement - QA

GUERRAOUI Ismail - SA

Sommaire :

I. Périmètre fonctionnel 4

II. Conception UML 6

1. Glossaire 6

- Logique restaurant : 6
- Logique plat (Dish) : 7
- Logique commande (Order) : 7
- Logique paiement : 8

2. Diagramme de cas d'utilisation 8

- Présentation des acteurs 8
- Principaux cas d'utilisation 8

3.Diagramme de classes 10

4. Design Patterns 10

- Diagramme de classe annoté : 10
  - Arbitrage et justification des choix : 12

5. Diagramme de séquence 13

6. Maquette 13

III. Qualité des codes et gestion de projets 16

Qualité des tests et coverage 16

Qualité code 16

Gestion du projet 17

IV. Auto-évaluation 18

- Ce qui a bien fonctionné 18
- Ce que nous avons appris 18
- Ce qui pourrait être amélioré 18
- Auto-évaluation du travail 18

I. Périmètre fonctionnel

Côté	Exigence	Etat	Remarques	Points forts	Points faibles
Client	C1	done		Architecture claire, données bien structurées	Accès direct aux données, sans contrôle d'accès ni couche de présentation
Client	C2	done		**Restaurant Filter** très complet et maintenable	Une fonction générale filter pas très propre
Client	C5	done		Bonne intégration avec la logique commande	Validation partielle des entrées utilisateur
Client	C6	done	Modification - visualisation & validation d'un créneau (réunio PO)	Visualisation et validation du créneau fonctionnelles via **Order.chooseSchedule() et RestaurantScheduleManager**	Gestion limitée aux cas simples, pas de vérification multi-jours ni retour d'erreur clair
Client	C7	Done		Processus de commande cohérent **(OrderManager, OrderState)**	Pas encore de gestion d'erreurs ou validations utilisateur
Restaurant	R2	Done		Structure métier claire et extensible	Risque de **NullPointerException** (liste non initialisée), validation d'entrée manquante
Restaurant	R4	Done	Système de suggestion	Suggestions automatiques	IA encore basique, pas

			automatique à la saisie par mots clés	fonctionnelles (R4a) avec fusion des toppings et mots-clés	de vérification de cohérence ni de priorisation intelligente
Restaurant	R5	Done		Création et mise à jour des créneaux par demi-heures	Code complexe et peu robuste
Paiement	P1	Done		Paiement déclenché via **OrderManager.proceedPayment() et PaymentService,** flux global cohérent	Paiement simulé (pas de service réel)
Paiement	P2	Done		Enregistrement automatique des paiements dans la commande (**PaymentState**)	Manque de logs et retours d'erreur
Paiement	P3	Done		Crédit étudiant géré avec **InternalPayment** et débit automatique du solde	Pas de rechargement possible Aucune vérification de remboursement ou échéance
Paiement	P6	Done		M ode de paiement clairement enregistré dans **Payment.getPaymentMethod()**	Le mode de paiement est stocké mais pas encore exploité

IA/Catalogue	AI2	BLOCKED	Avec R4, on propose déja des suggestions automatiques mais il n'y a pas de vérification de cohérence	L'IA suggère mais ne valide pas (contrôle humain implicite via R4)	Pas de contrôle automatique de cohérence ni de traçabilité des erreurs

II. Conception UML

1. Glossaire

- Logique restaurant :

Terme	Définition
FilterCriteria	Objet regroupant tous les filtres possibles (nom, prix min/max, type de cuisine, disponibilité, tags...) utilisés par **RestaurantFilter.**
RestaurantScheduleManager	Classe de service gérant la création, modification et validation des créneaux d'ouverture et de capacité.
RestaurantSchedule	Représente un créneau horaire (jour, heure de début, durée, capacité de commandes).
DayOfTheWeek	Enumération interne définissant les jours de la semaine utilisés dans les plannings (**RestaurantSchedule**).
CuisineType	E numération précisant le type de cuisine (italienne, française, etc.)
establishmentType	Enumération indiquant le type d'établissement (restaurant, food truck, CROUS, etc.)

- Logique plat (Dish) :

Terme	Définition
DishSuggestionEngine	Interface définissant la méthode commune suggest(String keyword) utilisée par les moteurs de suggestion de plats (ex. **KeywordSuggester**).
KeywordSuggester	Composant de l'IA de suggestion proposant un plat à partir de mots-clés saisis (ex. "salade" → "entrée végétarienne").
Suggestion	Structure représentant une proposition générée automatiquement.
DishFactory	Classe utilitaire pour créer des plats à partir d'entrées structurées
DishInput	Structure de saisie contenant les champs nécessaires à la création d'un plat.
SmartDishFactory	Variante de **DishFactory** intégrant la logique de suggestion intelligente basée sur l'historique.

- Logique commande (Order) :

Terme	Définition
OrderManager	Service qui gère le cycle de vie d'une commande : création, ajout/suppression de plats, validation et paiement.
OrderState	Enum représentant les différents états possibles d'une commande (**CREATED**, **PAID**, **IN_PREPARATION**, etc.).
OrderDeliveryPoint	Enumération listant les points de livraison disponibles sur le campus (ex. **MAIN_ENTRANCE**, **LIBRARY**, **CAFETERIA**, etc.), utilisés lors de la création d'une commande.
OrderNotFoundException / AlreadyExistingOrder	Exceptions gérant les cas d'erreur liés à la présence ou absence d'une commande en cours.

- Logique paiement :

Terme	Définition
InternalPayment	Paiement effectué via le crédit étudiant interne de l'utilisateur.
ExternalPayment	Paiement classique simulé (ex. carte, PayPal) via un service externe fictif.
PaymentState	Enum représentant l'état d'un paiement (en attente, en cours, validé, échoué).
NotEnoughMoney	Exception levée lorsqu'un utilisateur tente un paiement avec un crédit insuffisant.

2. Diagramme de cas d'utilisation

- Présentation des acteurs

User : utilisateur non authentifié, peut naviguer dans les restaurants et plats disponibles.

Campus Member : sous-type de *User*, il est authentifié et peut passer une commande complète (ajout d'items, choix du lieu, paiement, suivi).

Restaurant Owner : responsable d'un restaurant, il peut enregistrer son restaurant, ajouter/éditer ses plats et gérer ses disponibilités.

Payment System : service externe permettant d'effectuer et de confirmer les paiements des commandes.

- Principaux cas d'utilisation

Pour le **User** :

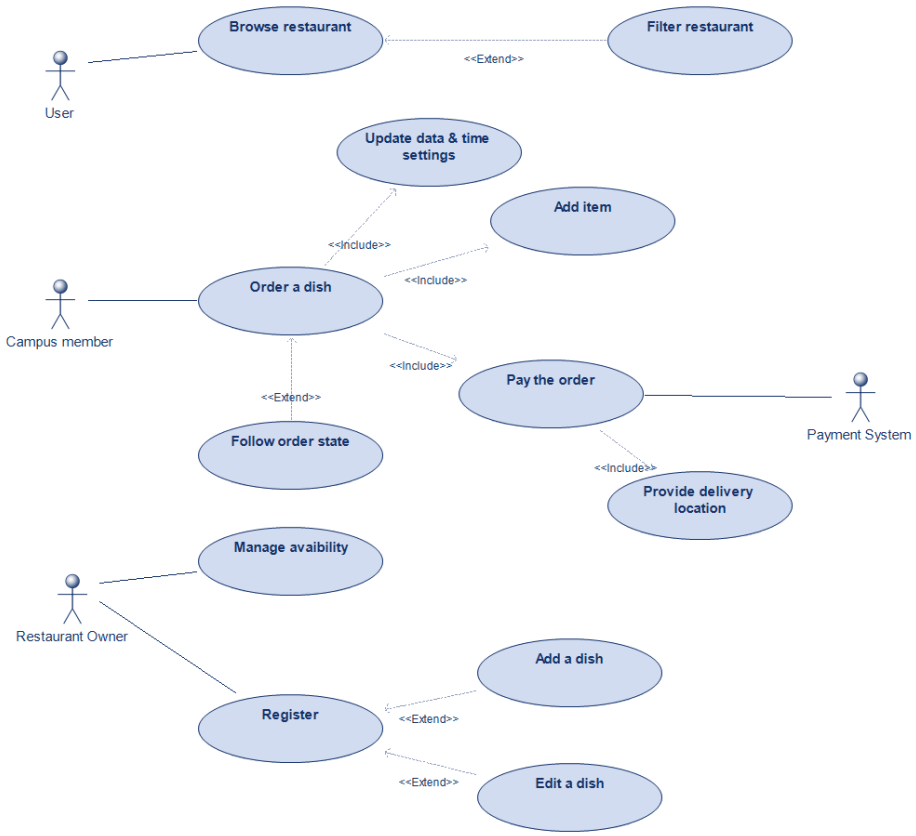
- **Browse restaurant** : consulter la liste des restaurants disponibles.
- **Filter restaurant** : appliquer des filtres (prix, type de cuisine, horaires, options alimentaires). (*Extend* de *Browse restaurant*).

Pour l'acteur **Campus Member :**

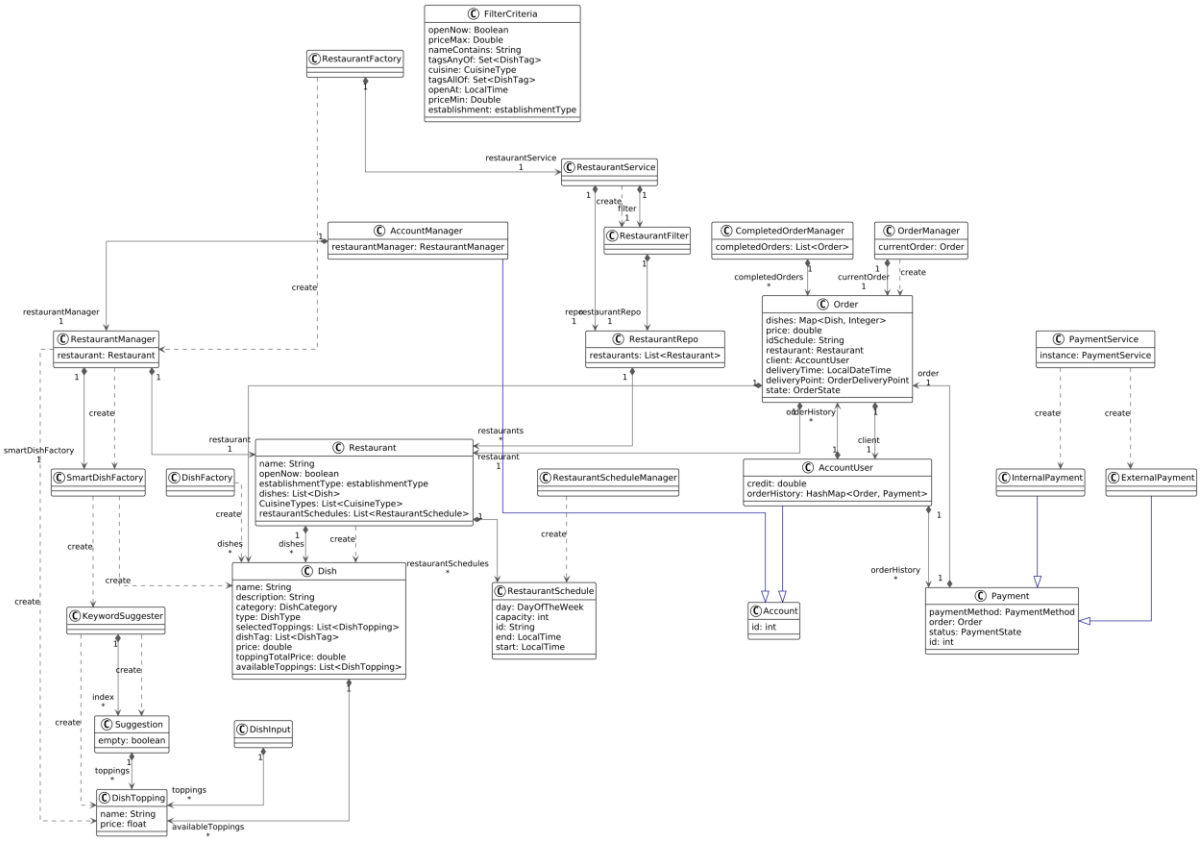
- **Order a dish** : créer une commande en sélectionnant un restaurant et des plats.
- *Include* :
    - **Add item** : ajouter un plat à la commande.
    - **Update data & time settings** : définir la date et le créneau de livraison possible.
    - **Provide delivery location** : choisir un lieu de livraison prédéfini.
    - **Pay the order** : effectuer le paiement (externe ou crédit étudiant).
- **Follow order state** : consulter l'avancement de sa commande (*Extend* de *Order a dish*).

Pour l'acteur **Restaurant Owner :**

- **Register** : inscrire un restaurant dans le système.
- *Extend* : **Add a dish** (ajout de nouveaux plats).
- *Extend* : **Edit a dish** (modifier les informations d'un plat).
- **Manage availability** : définir ou modifier les créneaux horaires et la capacité (nb max de commandes par demi-heure).

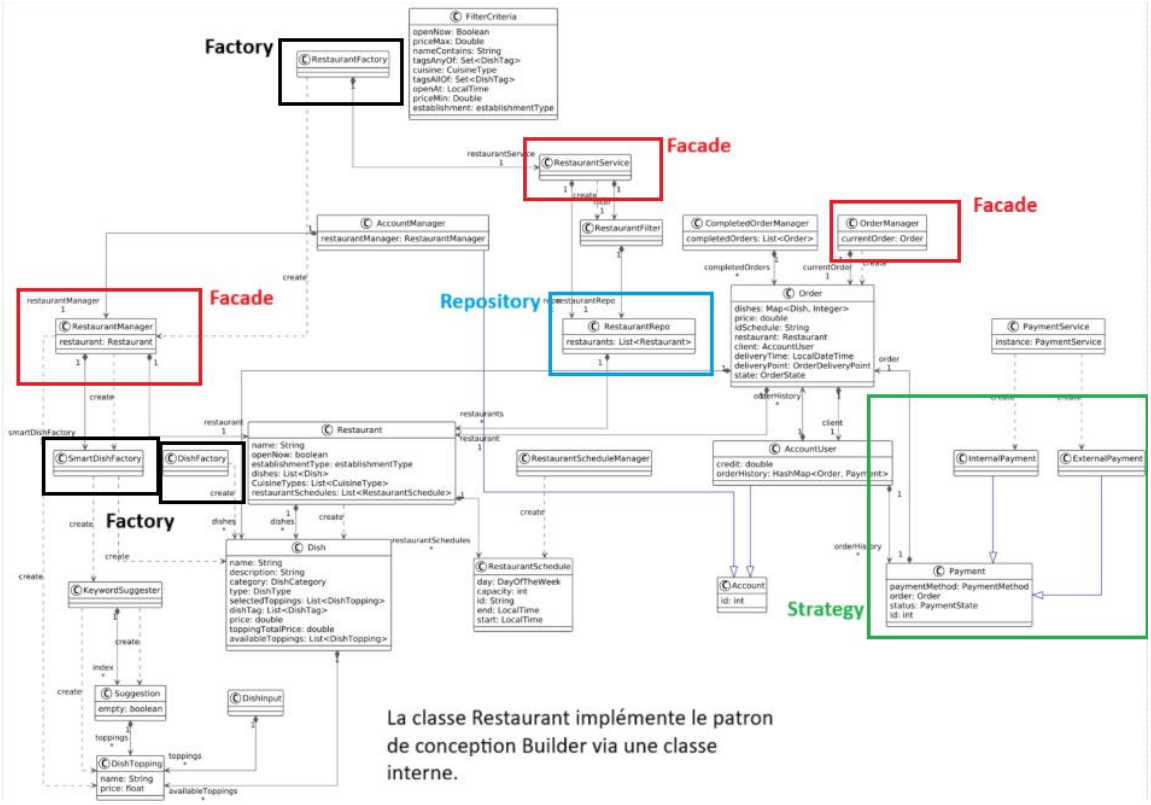


3.Diagramme de classes



4. Design Patterns

Diagramme de classe annoté :



La classe Restaurant implémente le patron de conception Builder via une classe interne.

Arbitrage et justification des choix :

- Patrons présents :

  RestaurantManager (Facade) : Nous avons choisi le patron Façade pour le RestaurantManager afin de bien séparer les responsabilités. Il joue le rôle de chef d'orchestre : il ne fait pas le travail lui-même, mais il sait à qui s'adresser. Il s'occupe donc de faire appel aux bonnes classes selon la situation et de gérer les erreurs de manière centralisée, en accord avec les principes GRASP.

  RestaurantService (Facade) : Cette classe utilise le patron Façade pour cacher la complexité des autres classes (en l'occurrence, le RestaurantFilter et le RestaurantRepo). Elle permet donc d'avoir un accès simplifié aux différents éléments et facilite son utilisation, notamment dans le futur pour l'interface visuelle.

  OrderManager (Facade) : Ce patron est utilisé ici pour servir de point de contact unique pour tout le processus de commande. Il cache la complexité des interactions entre la commande, le système de paiement et la gestion des horaires du restaurant, ce qui simplifie radicalement l'utilisation pour le reste du code.

  Payment (Strategy) : Ce patron nous permet de définir plusieurs algorithmes de paiement et de les rendre interchangeables. Chaque méthode devient une stratégie isolée, ce qui permet au PaymentService de choisir la bonne à la volée sans en connaître les détails, rendant le système facile à étendre.

  RestaurantRepo (Repository) : Ce patron centralise la manière dont on stocke notre collection de restaurants.

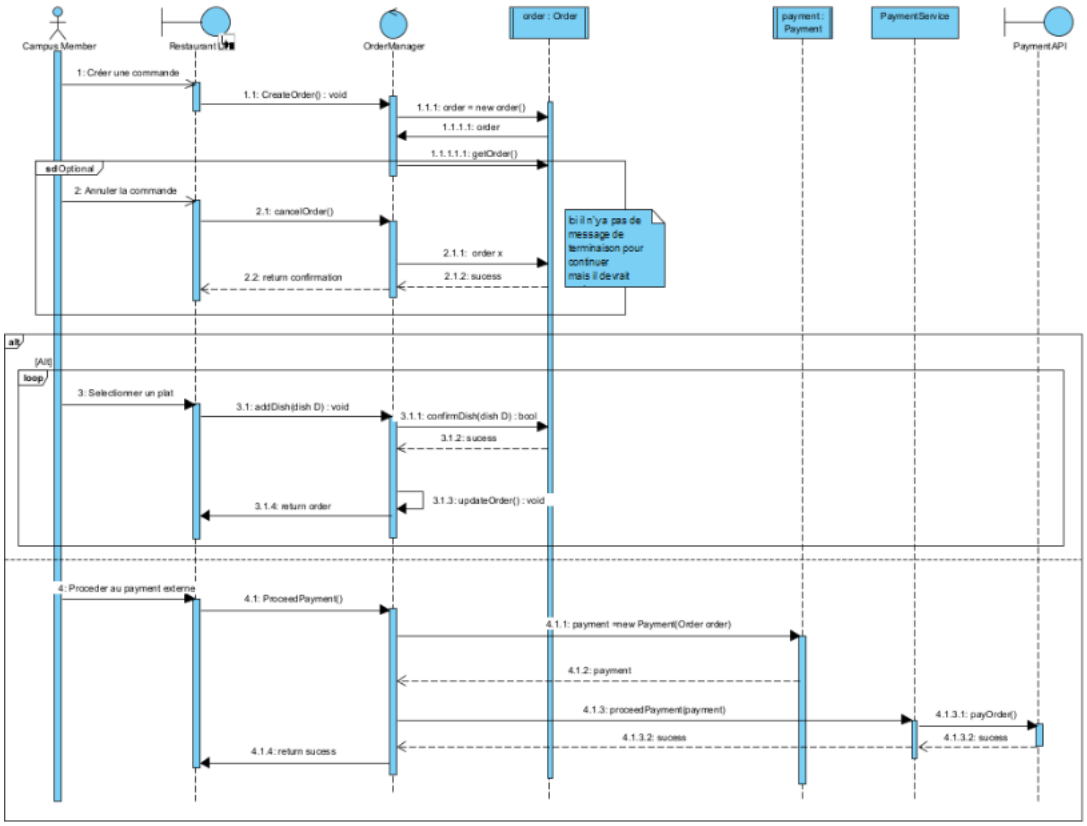
  RestaurantFactory (Factory) : Ce patron a été choisi après avoir abandonné la piste du Singleton pour le RestaurantManager. On l'a donc choisi pour que le processus de création du restaurant ne soit pas fait dans une classe qui n'a pas à avoir accès aux différents services nécessaires. Il nous permet de créer le restaurant, son RestaurantManager associé, et de l'ajouter dans le repo à travers le RestaurantService, le tout en une seule étape.

Restaurant (Builder) : Ici, on a choisi de faire un Builder pour le restaurant afin d'éviter de créer trop de constructeurs, ou des constructeurs avec trop de paramètres. Avec le Builder, on en a un unique et personnalisable comme on veut.

- Patrons pas présents :

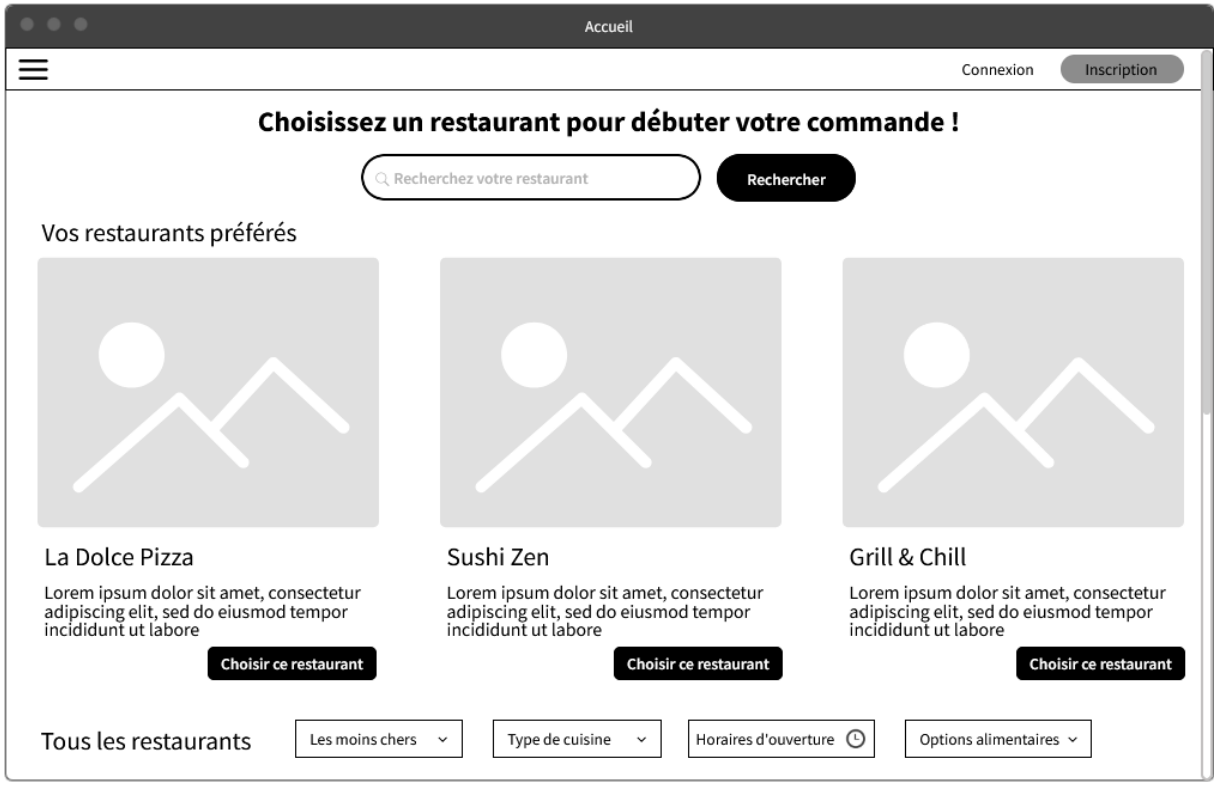
RestaurantManager (Singleton) et RestaurantService (Singleton) : On a envisagé à un moment de faire en sorte que nos différentes façades soient des Singletons. On se demandait en effet comment, lors de la création d'un restaurant, on pouvait lui associer son RestaurantManager. On s'était dit que le RestaurantManager pouvait être unique et recevoir uniquement des demandes. Mais si on faisait ça, on ne respectait pas bien les principes de la POO. On a donc abandonné l'idée et on s'est redirigé vers une Factory.

5. Diagramme de séquence

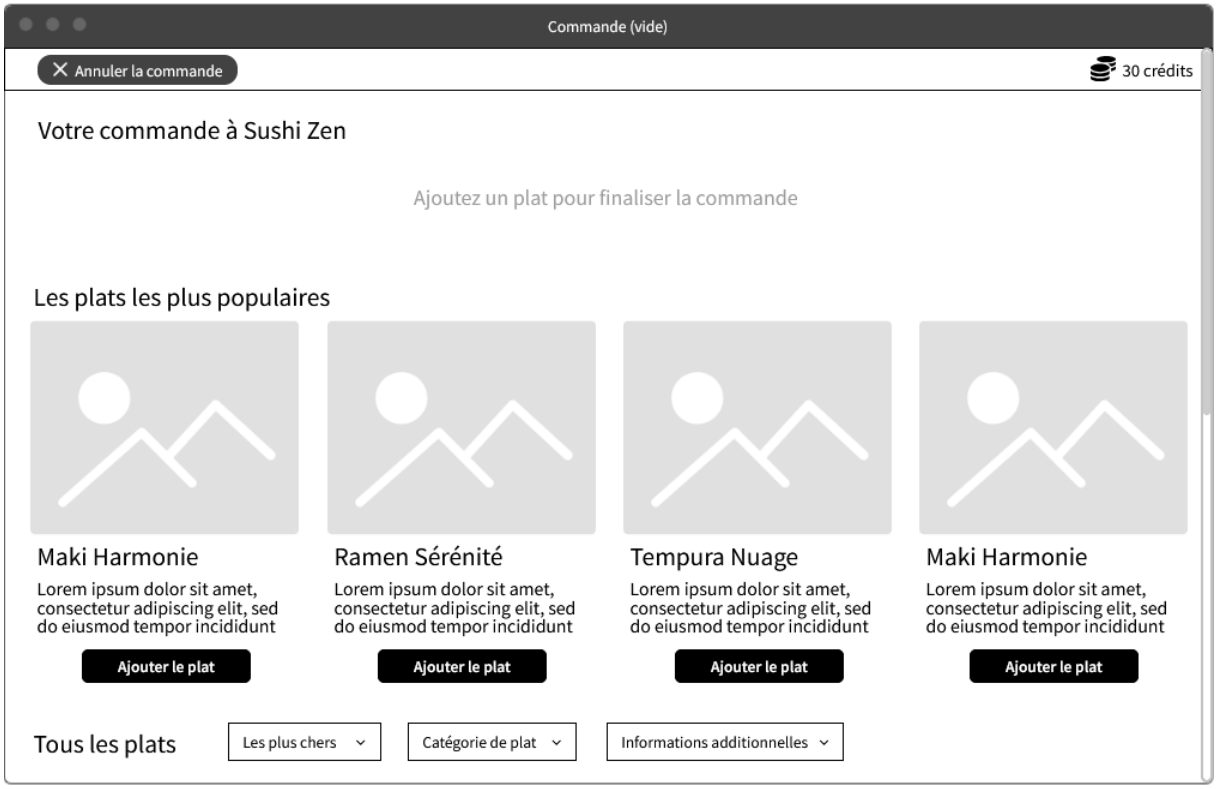


6. Maquette

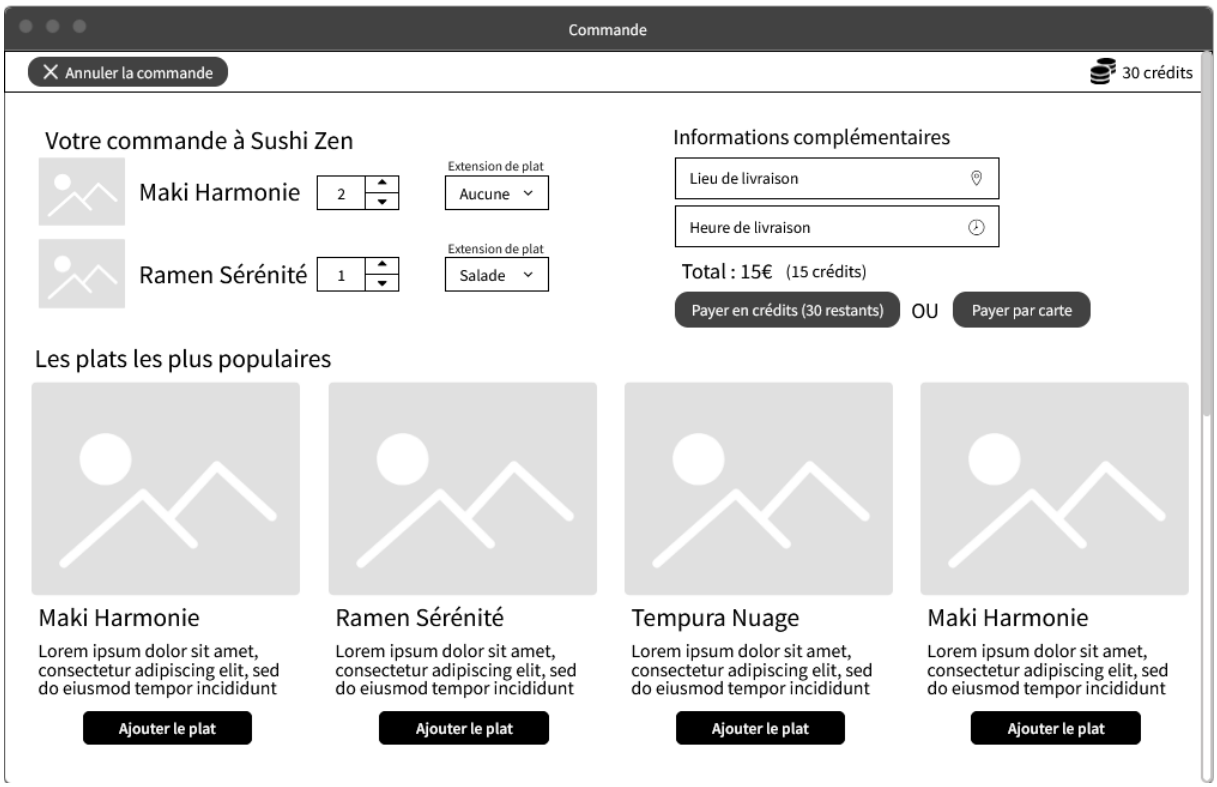
Accueil



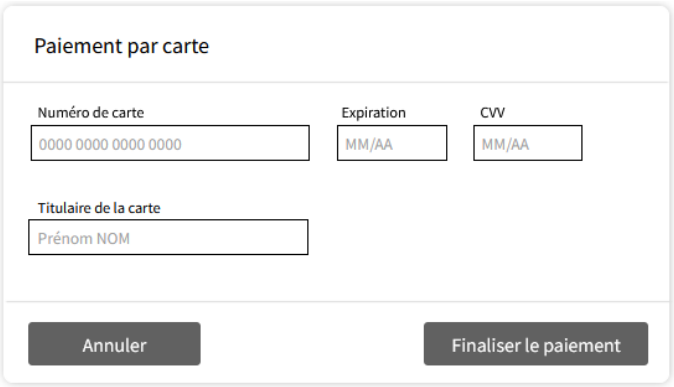
Commande (vide)



Commande



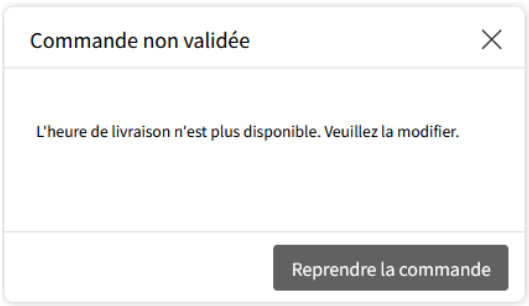
Paiement



Popup annuler la commande



Popup heure de livraison indisponible



III. Qualité des codes et gestion de projets

Qualité des tests et coverage

Périmètre et type des tests :

-Unitaires (JUnit/Mockito) : ils testent les classes principales du projet en testant le comportement isolé de ces classes. Nous n'avons pas jugé nécessaire de tester certaines classes, comme des enum ou des classes modélisant des données (ex : Client) car il n'y a pas besoin de s'assurer de comportements de getters ou setters.

-Intégration (Cucumber) : Les démonstrations principales, pouvant s'apparanter à des scénarios E2E, à savoir le choix d'un plat dans un restaurant donné puis le paiement de cette commande et l'ajout d'un restaurant dans la base de données ont été réalisés avec Cucumber. Cela permet de définir un enchainement des actions clair et une certaine modulabilité dans les données qu'on donne en test. Nous avons aussi pu effectuer des tests d'intégration pour le filtre de restaurants par exemple.

-Couverture :

Grâce à une analyse SonarCube (annexe 1), on peut voir que les classes centrales au métier ont un coverage assez convenable, autour de 70%, nombre certainement dû au fait que beaucoup de ces classes contiennent des getters/setters et qu'il n'est pas nécessaire de les tester.

Qualité code

Le code repose assez largement sur une architecture inspirée du pattern Service/ Manager où l'on peut distinguer :

- Les entités **(`Restaurant`, `Dish`, `Order`)** immuables
- Les **managers** (`OrderManager`, `RestaurantManager`) encapsulent la logique métier et servent d'interface propre avec le reste du système (ex. commandes, réservations).
- Les **repositories** gèrent la persistence des données

Les managers n'agissent pas comme "God Class", ils ont chacun une responsabilité unique, comme le montre le découpage de la gestion des restaurants en RestaurantScheduleManager qui s'occupe de gérer la capacité d'un restaurant et RestaurantManager qui s'occupe de la gestion des plats pour un restaurant donné.

Gestion du projet

Le projet est versionné sous **Git** avec une organisation légère inspirée de GitFlow :

- **`main`** est une branche semi-stable où les Pull Request s'effectuent.
- **Branches `feature/`** pour les développements courts qui seront ensuite
- **Production** contient les versions stables

Chaque Pull Request déclenche la CI qui pour l'instant ne fait que run tous les tests grâce à la commande mvn test. Elle ne peut être merge que si tous les tests run.

IV. Auto-évaluation

Ce qui a bien fonctionné

- Sur le plan technique :
  - → Chaque membre du groupe à fournit du code testé avant de push, du code de qualité en suivant des patrons de conception, mais aussi en respectant le principe de la POO.
  - → La structure initiale à était respecté et si changement il devait y avoir, communication il y a eu.

- Sur le plan de l'organisation et de la communication :
  - → Chaque membre de l'équipe a rédigé ses propres User Stories et tâches associées. Ce travail a été réalisé de manière collaborative, en se réunissant afin de discuter. Un tableau Kanban clair et structuré a aussi été mis en place.
  - → Personne hésiter à parler, donner son avis ou poser des questions.
  - → Les délais étaient respectés.

Ce que nous avons appris

- Compétences techniques :
  - → Utilisation de Cucumber. Apprentissage de plusieurs patron de conception
- Méthodologie de projet :
  - → Utilisation renforcée du Kanban
  - → Revue du code avant chaque pull request (en provenance des branches feature)

Ce qui pourrait être amélioré

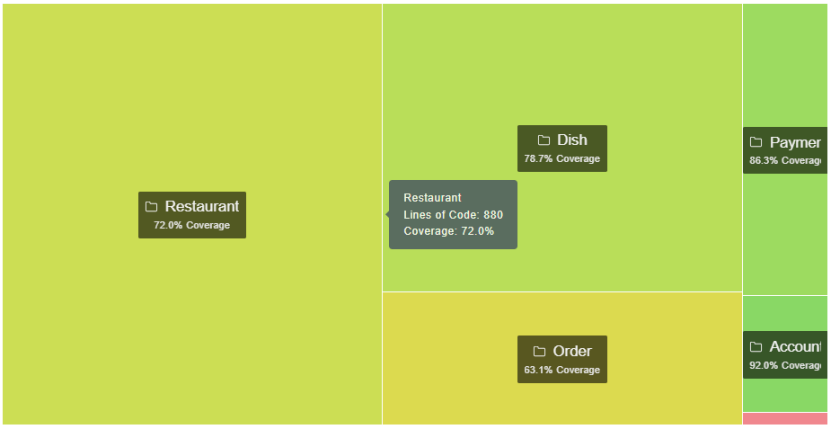
→ Mise en place d'une démonstration plus tôt, et l'enrichir au fur et à mesure
→ Mettre à jour à chaque changement le diagramme de classe et séquence.
→ Dans l'ordre des User Story implémentées, avoir des US traitant de classe ayant un lien entre eux dans l'optique de développer un scénario et donc plus largement une démo plus tôt plutôt que des bouts de codes n'interagissant pas entre eux et qui nécessitaient tous d'être finis pour avancer

Auto-évaluation du travail

☺ MADANDAR Arani Ali : 100
☺ BOULET Olivier : 100
☺ MENOU Clément : 100
☺ GUERRAOUI Ismail : 100
☺ KARBAL Adam : 100

Total : 500

Annexes :



Annexe 1 : Analyse SonarCube du coverage des tests du projet.